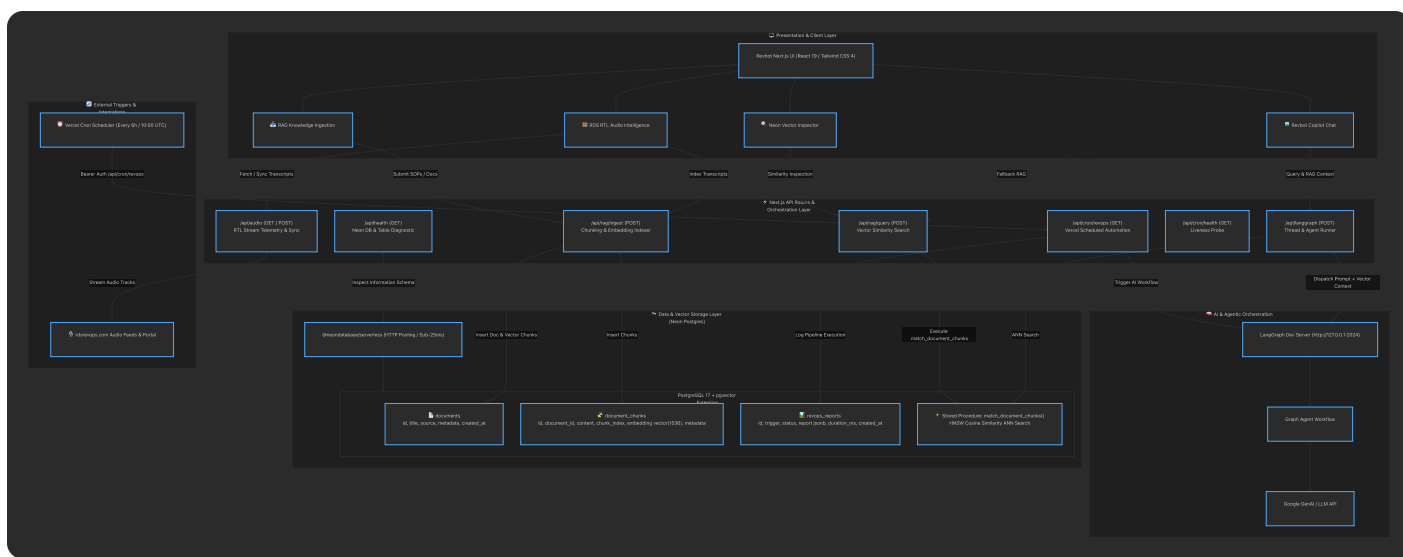


Production RevOps AI Assistant & Vector Knowledge Engine powered by **Next.js 16**, **Neon Serverless Postgres (pgvector)**, **LangGraph Orchestration**, and **Real-Time RTL Audio Processing**.



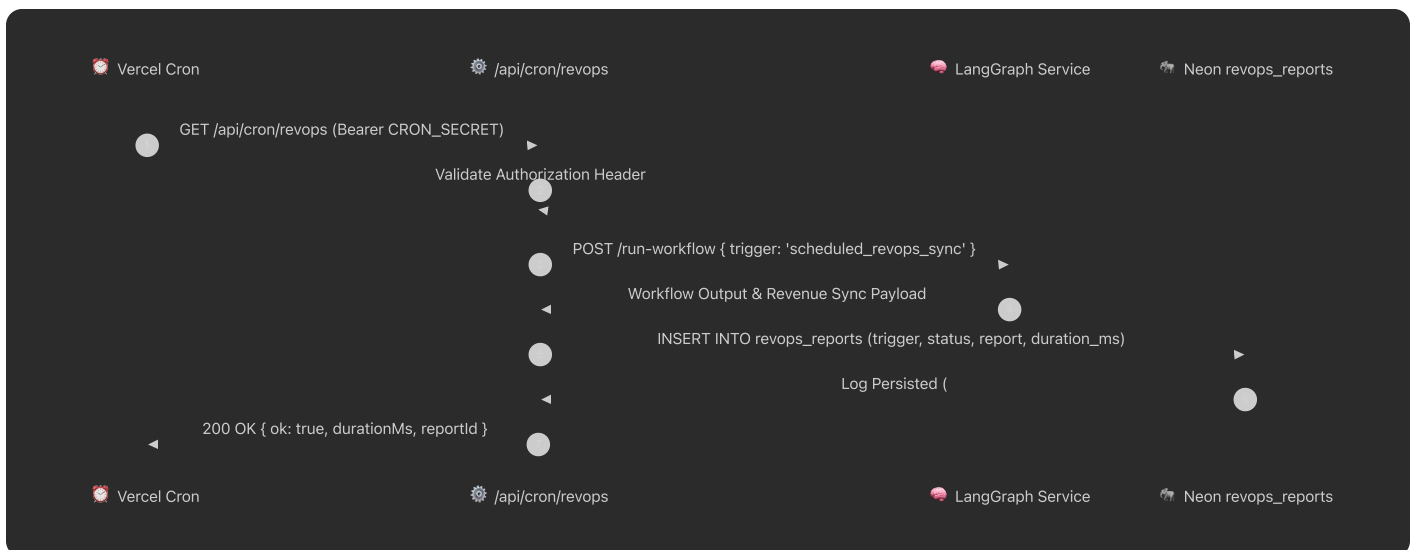
1. Vector Search & RAG Flow (Revbot Copilot)



2. Knowledge Ingestion & Chunking Pipeline



3. Vercel Cron Automated Pipeline



Database Topography & Schema Specifications

documents			
serial	id	PK	
text	title		Document Title
text	source		File origin / URL
jsonb	metadata		Category, author, tags
timestampz	created_at		Timestamp

revops_reports			
serial	id	PK	
text	trigger		scheduled_revops_sync / manual
text	status		completed / failed
jsonb	report		AI workflow result payload
integer	duration_ms		Execution time in ms
timestampz	created_at		Timestamp

contains (1:N)



document_chunks			
serial	id	PK	
integer	document_id	FK	References documents(id)
text	content		Text Chunk (300 words)
integer	chunk_index		Sequence index
vector_1536	embedding		HNSW Cosine Vector(1536)
jsonb	metadata		Chunk-level tags



API Route Catalog

Endpoint	Method	Purpose	Auth / Protection
<code>/api/langgraph</code>	<code>POST</code>	Dispatches user queries with Neon RAG context to the LangGraph dev server	None (Public UI)
<code>/api/rag/query</code>	<code>POST</code>	Performs direct <code>pgvector</code> HNSW similarity queries	None (Public UI)

Endpoint	Method	Purpose	Auth / Protection
<code>/api/rag/ingest</code>	<code>POST</code>	Chunks arbitrary documents, generates vector embeddings, and stores in Neon	None (Internal)
<code>/api/audio</code>	<code>GET</code> , <code>POST</code>	Fetches real-time RTL audio tracks & syncs audio transcripts	None (Public/Internal)
<code>/api/cron/revops</code>	<code>GET</code>	Executes automated RevOps AI pipeline every 6h and saves to <code>revops_reports</code>	<code>Bearer</code> <code>CRON_SECRET</code>
<code>/api/cron/health</code>	<code>GET</code>	Liveness health check for cron scheduler	None
<code>/api/health</code>	<code>GET</code>	Neon database connection diagnostic & table inspection	None

Tech Stack & Runtime Components

Layer	Technology	Version / Spec
Framework	Next.js (App Router)	<code>16.3.3</code>
Frontend	React / React DOM	<code>19.2.4</code>
Styling	Tailwind CSS / Oxide Darwin ARM64	<code>4.3.3</code>
Database	Neon Serverless PostgreSQL	<code>PostgreSQL 17</code>
Vector Engine	<code>pgvector</code> HNSW (<code>cosine_ops</code>)	<code>1536 Dimensions</code>
DB Driver	<code>@neondatabase/serverless</code>	<code>1.1.0</code> (HTTP connection pool)
Agent Orchestration	LangGraph SDK	<code>@langchain/langgraph-sdk</code> <code>1.9.28</code>
AI / GenAI	<code>@google/genai</code>	<code>2.19.0</code>
Deployment	Docker / Vercel Edge & Serverless	Linux / ARM64 multi-stage



Getting Started

1. Environment Variables

Create a `.env.local` file in the project root:

```
# Neon Postgres Database Connection URLs - Medallion Multi-Tier Architecture
DATABASE_URL=postgresql://user:password@ep-yellow-fog-a8xjbhox-pooler.eastus2.
NEON_DB_GOLD_URL=postgresql://user:password@ep-yellow-fog-a8xjbhox-pooler.east
NEON_DB_SILVER_URL=postgresql://user:password@ep-sample-silver-pooler.eastus2.
NEON_DB_BRONZE_URL=postgresql://user:password@ep-sample-bronze-pooler.eastus2.
AIRO_BUILDER_ORIGIN=https://rdsrevops.com

# LangGraph & Cron Automation
LANGGRAPH_API_URL=http://127.0.0.1:2024
LANGGRAPH_API_KEY=optional_key_if_configured
CRON_SECRET=your_vercel_cron_secret
```

2. Local Development

```
# Install dependencies
npm install

# Run Next.js development server
npm run dev

# Run automated RAG & LangGraph endpoint tests
node scripts/test-rag.mjs
node scripts/test-langgraph-endpoint.mjs
node scripts/test-catalog.mjs
```

3. Docker Build & Run

```
docker compose up -d
```



Agent Setup & Configuration Guide (Neon, LangGraph & RevOps MCP)

This section provides standardized instructions for AI Agents (Antigravity, Cursor, Claude Code, Windsurf) to bootstrap, verify, and orchestrate the **Neon Postgres Medallion Database**, **LangGraph Dev Server**, and **RevOps MCP Server**.

1. Neon Serverless Postgres Setup

The platform uses a **Medallion Architecture (Gold / Silver / Bronze)** powered by Neon Postgres 17 with **pgvector** for high-dimensional vector search.

A. Database Schema Bootstrap DDL

Agents should ensure the following DDL is executed on the target Neon database:

```
-- 1. Enable pgvector extension
CREATE EXTENSION IF NOT EXISTS vector;

-- 2. Master Documents Table
CREATE TABLE IF NOT EXISTS documents (
    id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    source TEXT,
    metadata JSONB DEFAULT '{} '::jsonb,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Document Vector Chunks with 1536-D HNSW Index
CREATE TABLE IF NOT EXISTS document_chunks (
    id SERIAL PRIMARY KEY,
    document_id INTEGER REFERENCES documents(id) ON DELETE CASCADE,
    content TEXT NOT NULL,
    chunk_index INTEGER NOT NULL,
    embedding VECTOR(1536),
    metadata JSONB DEFAULT '{} '::jsonb,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```

CREATE INDEX IF NOT EXISTS document_chunks_embedding_hnsw_idx
ON document_chunks USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- 4. RevOps Execution Reports Table
CREATE TABLE IF NOT EXISTS revops_reports (
    id SERIAL PRIMARY KEY,
    trigger TEXT NOT NULL,
    status TEXT NOT NULL,
    report JSONB NOT NULL,
    duration_ms INTEGER,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 5. Stored Procedure for Cosine ANN Vector Search
CREATE OR REPLACE FUNCTION match_document_chunks (
    query_embedding VECTOR(1536),
    match_threshold FLOAT,
    match_count INT
)
RETURNS TABLE (
    id INT,
    document_id INT,
    content TEXT,
    metadata JSONB,
    similarity FLOAT
)
LANGUAGE sql STABLE
AS $$
    SELECT
        document_chunks.id,
        document_chunks.document_id,
        document_chunks.content,
        document_chunks.metadata,
        1 - (document_chunks.embedding <=> query_embedding) AS similarity
    FROM document_chunks
    WHERE 1 - (document_chunks.embedding <=> query_embedding) > match_threshold
    ORDER BY document_chunks.embedding <=> query_embedding
    LIMIT match_count;
$$;

```

2. LangGraph Dev Server Setup

The `/api/langgraph` endpoint connects to a local or remote LangGraph execution runtime.

A. Install LangGraph CLI

```
pip install -U "langgraph-cli[inmem]"
```

B. Run LangGraph Dev Server

Start the LangGraph runner on port `2024`:

```
langgraph dev --port 2024 --host 127.0.0.1
```

Verify the server is running:

```
curl http://127.0.0.1:2024/ok
```

C. Validate LangGraph Endpoint Integration

Run the bundled integration test script to verify Neon RAG retrieval and LangGraph dispatch:

```
node scripts/test-langgraph-endpoint.mjs
```

3. RevOps & Neon MCP Server Configuration

To grant AI Agents direct tool access to Neon Postgres and RevOps operations, configure the following in your IDE's `mcp_config.json` (e.g. `~/.gemini/config/mcp_config.json` or `.agents/mcp_config.json`):

```
{
  "mcpServers": {
    "mcp-server-neon": {
      "command": "npx",
      "args": [
        "-y",
        "mcp-remote",
```



```

    "https://mcp.neon.tech/mcp"
  ],
  "env": {}
},
"revops-postgres-mcp": {
  "command": "npx",
  "args": [
    "-y",
    "@modelcontextprotocol/server-postgres",
    "postgres://user:password@ep-yellow-fog-a8xjbhox-pooler.eastus2.azure"
  ],
  "env": {}
}
}
}

```

Available Agent MCP Tools

- **mcp-server-neon**:
 - **run_sql** / **run_sql_transaction**: Run arbitrary queries and DDL migrations.
 - **describe_table_schema**: Inspect database structures and constraints.
 - **create_branch** / **reset_from_parent**: Manage database branches for isolated agent scratchpads.
- **revops-postgres-mcp**:
 - **query**: Execute direct vector queries and audit log queries against **documents**, **document_chunks**, and **revops_reports**.
 - **schema**: Inspect table definitions and column types in real-time.

4. Verification Checklist for Agents

Before concluding any architectural or code change, run:

```

# 1. Verify ESLint compliance
npm run lint

# 2. Verify Next.js build and route compilation
npm run build

# 3. Test RAG vector ingestion & HNSW cosine similarity search

```

```
node scripts/test-rag.mjs
```

```
# 4. Test LangGraph endpoint mock & thread dispatch
```

```
node scripts/test-langgraph-endpoint.mjs
```

```
# 5. Check multi-tier database health diagnostic
```

```
curl http://localhost:3000/api/health
```